OD

Oracle DBA Copilot

AI-powered Oracle diagnostics

Username

admin

Password

••••••••

Sign in

F-01 · AWR Report Analyzer

Upload AWR HTML/text → full diagnosis + action plan

Paste Text | Upload File

UPLOAD FILE

awrrpt_dbrac1_28_29.html

Remove file

Max 20MB · AWR HTML, SQL, TXT, LOG, CSV supported

QUESTION (OPTIONAL)

Perform a complete AWR performance diagnosis. Structure your output as:
1. Executive Summary (2 sentences for management, 2 for DBA)
2. Top wait events — classify as CPU / I/O / Concurrency / Network / RAC
3. Top 5 SQL statements by elapsed time — explain why each is slow
4. Memory health (buffer cache hit %, shared pool)
5. I/O health (read/write rates, wait time contribution)
6. Top 3 actionable recommendations, ranked by expected improvement

Use traffic-light icons 🔴 🟡 🟢 for severity. Include the exact SQL_IDs and wait event names from the report.

Leave blank to use the default analysis prompt for this module.

Run Analysis

QUESTION

Perform a complete AWR performance diagnosis. Structure your output as:
1. Executive Summary (2 sentences for management, 2 for DBA)
2. Top wait events — classify as CPU / I/O / Concurrency / Network / RAC
3. Top 5 SQL statements by elapsed time — explain why each is slow
4. Memory health (buffer cache hit %, shared pool)
5. I/O health (read/write rates, wait time contribution)
6. Top 3 actionable recommendations, ranked by expected improvement

Use traffic-light icons 🔴 🟡 🟢 for severity. Include the exact SQL_IDs and wait event names from the report.

📋 AWR Performance Diagnosis — DBRAC / dbrac1 / Snaps 28–29

Instance: DBRAC | **Host:** rac1 (Linux x86 64-bit) | **Period:** 17-Dec-2025 10:14–10:25 (11.41 min) | **Oracle:** 11.2.0.4.0 RAC (2 nodes, 2 CPUs)

📋 Executive Summary

For Management (plain language)

The database is spending almost all of its time running a single poorly-written query that reads an entire large table **3,000 times in 11 minutes** — instead of using a shortcut (index). This one query is responsible for over **92% of all database activity** and is consuming the equivalent of both available processors at full speed. The single most important fix is: **add a proper index to the `TEST_SALES` table** — estimated 15–30 minutes of DBA work, with an expected improvement of 95%+ in query runtime.

For DBA (technical)

Total DB Time = **149.6 seconds** over 11.41 minutes (AAS = 0.22 — lightly loaded overall, but CPU-saturated when the bad SQL runs). Dominant wait: **DB CPU at 83.53% of DB Time** — classic CPU-bound profile. Worst offender: SQL_ID **`9u9w5ykpwumfs`** (`SELECT COUNT(*) FROM TEST_SALES WHERE UPPER(CUSTOMER_NAME)...`) — 3,000 executions, **9,821,800 buffer gets**, 137.35s elapsed, 0 disk I/O — a full table scan (reading entire table, no shortcut/index) driven by a non-sargable predicate (`UPPER()` function on an unindexed column). Parent PL/SQL wrapper: SQL_ID **`9g4v8sh43hxdv`** .

🚦 Health Scorecard

Area	Status	Key Metric	Plain Explanation
CPU / Query Logic	🔴 Critical	83.53% of DB Time is DB CPU	One query is burning both CPUs doing unnecessary brute-force table reads
Memory (Buffer Cache)	🟢 OK	Buffer Hit = 99.99% (Pool Hit 100%)	All data is being served from memory — no disk pressure from the bad query
Shared Pool / Library Cache	🟡 Warning	Library Hit = 87.59%; Soft Parse = 80.71%	About 1 in 8 query lookups misses the plan cache; hard parse rate is elevated at 17.9/txn
Disk I/O	🟢 OK	db file sequential read = 0.32% DB Time; Physical reads = 1.23/sec	Disk is not the bottleneck — the problem is CPU, not storage
RAC / Concurrency	🟡 Warning	row cache lock = 1.89% DB Time; gc current block busy = 0.30%	Minor inter-node contention visible — acceptable but worth monitoring
Commit / Write-to-disk	🟢 OK	log file sync = 0.02% DB Time; 4 commits total	Almost no commit activity — workload is read-heavy
Undo / Snapshot Too Old risk	🟢 OK	STO = 0; Max Query Length = 318s; Tuned Retention = 19.4 min	No ORA-01555 risk in this window; undo is healthy

🔍 Top 5 Root Causes (ranked by impact)

🔴 Root Cause 1: Catastrophic Full Table Scan — Non-Sargable Predicate on TEST_SALES

Plain explanation: A query is searching a large sales table 3,000 times, but because it wraps the search column in a `UPPER()` function, Oracle cannot use any index — it must read every single row every single time. This is like searching a phone book by reading every entry instead of jumping to the right letter. It consumed 92% of all database work in this 11-minute window.

Technical detail: SQL_ID `9u9w5ykpwumfs` — `SELECT COUNT(*) FROM TEST_SALES WHERE UPPER(CUSTOMER_NAME) = 'CUSTOMER 500' AND AMOUNT > 500` — full table scan (reading entire table, no shortcut/index) on `DEMO_USER.USERS.TEST_SALES`. 3,000 executions × 3,273.93 buffer gets/exec = **9,821,800 logical reads** (99.83% of all logical reads in the snapshot). CPU: 121.99s (97.64% of total DB CPU). %IO = 0.00% — entirely in-memory brute force.

Evidence: Elapsed time = **137.35s** (91.83% of total DB Time 149.6s). Buffer gets: 9,821,800 out of total 9,838,384 = **99.83%**. Segment `TEST_SALES` = 9,824,560 logical reads = **99.86% of all logical reads**.

Business impact: Any application or batch job calling this query 3,000 times will complete ~45 seconds per loop iteration (137s / 3 PL/SQL calls). If this runs in production under load, it will starve every other user of CPU time. At 3,000 executions per 11 minutes, this is ~4.4 queries/second burning 100% of available CPU.

Confidence: High — SQL text, execution count, buffer gets, segment stats, and CPU consumption all corroborate each other unambiguously.

🔴 Root Cause 2: PL/SQL Loop Wrapper Amplifying the Bad Query 10×

Plain explanation: The bad query isn't being called once — it's being called inside a loop that runs it 10 times per invocation, and that invocation itself is called 3 times. This multiplier effect (10 × loop × 3 calls = 3,000 executions of the child SQL) means a design flaw in the code is amplifying the underlying index problem by 10-fold.

Technical detail: SQL_ID `9g4v8sh43hxdv` — `BEGIN FOR i IN 1..10 LOOP run_bad_query; END LOOP; END;` — 3 executions, 137.68s elapsed, 45.89s per exec. This is the PL/SQL wrapper calling `9u9w5ykpwumfs` 1,000 times per execution (3 × 1,000 = 3,000 child SQL executions). 9,825,075 buffer gets attributed at PL/SQL level.

Evidence: Elapsed time = **137.68s** (92.05% DB Time). CPU = 122.26s (97.86% of total DB CPU). PL/SQL execution elapsed time = 1.95s (1.30% DB Time) — the PL/SQL overhead itself is trivial; it's the SQL inside the loop that kills performance.

Business impact: Even after adding an index, the loop structure should be reviewed — the procedure calls `run_bad_query` in a loop which may itself be architecturally problematic.

Confidence: High — SQL text is explicit (`FOR i IN 1..10 LOOP run_bad_query`), execution counts match perfectly (3 × 1,000 = 3,000 child SQL calls).

🟡 Root Cause 3: Row Cache Lock Contention (Dictionary Cache — RAC)

Plain explanation: When the database needed to look up internal "dictionary" information (metadata about tables, sequences, etc.) shared across the two RAC nodes, sessions had to wait in line. This is like two offices sharing one filing cabinet — one office has to wait while the other is using it.

Technical detail: Wait event `row cache lock` — 1,257 waits, 2.8s total wait, avg 2ms, **1.89% of DB Time**. Background: `row cache process` = 16,545 events (0.55% bg time). RAC dictionary cache (`dc_histogram_defs` 70.46% miss, `dc_histogram_data` 36.36% miss, `dc_segments` 33.08% miss, `dc_sequences` 100% miss). GES requests for `dc_histogram_defs` = 1,221.

Evidence: `row cache lock` = 1.89% DB Time. Dictionary cache miss rates: `dc_histogram_defs` = **70.46%** (very high), `dc_sequences` = **100%**. `dc_object_grants` = 50%.

Business impact: Adds latency to session startup, query compilation, and DDL operations. At 179.57 waits/transaction, this is frequent. In a busier system, this could compound significantly.

Confidence: Medium — elevated but not alarming in absolute terms (2.8s total); dictionary cache misses are high, which is consistent with a freshly started or under-sized shared pool for the workload mix.

🟡 Root Cause 4: Hard Parse Overhead — Elevated Query Compilation Cost

Plain explanation: Every time Oracle receives a SQL query, it first checks if it has already figured out the best way to run it (soft parse = fast). If not, it has to figure it out from scratch (hard parse = slow, CPU-intensive). The system is doing hard parses at a rate that is consuming 2.22% of DB Time — noticeable but not catastrophic.

Technical detail: Hard parse elapsed time = **3.32s (2.22% of DB Time)**. Hard parses = 125 total (17.86/transaction — high). Parse CPU to Parse Elapsed % = **29.13%** — meaning 70.87% of parse time is spent waiting, not computing. Library cache hit = 87.59%. `SQL AREA` namespace: Get Pct Miss = 61.56%, Pin Pct Miss = 6.35%. `TABLE/PROCEDURE` namespace: Pin Pct Miss = 37.25%.

Evidence: Hard parse rate = 0.18/sec. Shared pool memory usage grew from 68.31% → 73.86% during the snapshot. Free memory in shared pool dropped from 69.26MB → 56.98MB (−17.74%). `shared pool` latch: 10 misses, 9 sleeps.

Business impact: Hard parses serialise through the shared pool latch and library cache, adding CPU and latency per new SQL statement. If literal SQL (non-bind-variable) is being used, the shared pool will fragment over time.

Confidence: Medium — the numbers are elevated but the snapshot is short (11 min) and involves AWR snapshot-taking activity which itself generates many unique SQL statements.

🟡 Root Cause 5: RAC Cluster Interconnect Overhead (gc current block busy / gc current block 2-way)

Plain explanation: In this two-node cluster, when one node needs a data block that the other node has modified, it must request it over the network interconnect. Some of these requests are experiencing delays because the sending node is busy — adding latency on top of the already CPU-heavy workload.

Technical detail: `gc current block busy` = 45 waits, 0.4s, avg **10ms**, 0.30% DB Time. `gc current block 2-way` = 180 waits, avg 2ms, 0.29% DB Time. `gc current grant busy` = 102 waits, avg 3ms, 0.17% DB Time. Global Cache Transfer: data blocks from Instance 2 — 80.71% immediate, **12.14% busy, 7.14% congested** (busy+congested = 19.28% — moderately elevated). Avg current block receive time = 4.5ms. Interconnect ping latency Instance 2 = avg **8.79ms** (500B), 10.08ms (8K).

Evidence: Total Cluster wait class = 1.22% DB Time (1.6% in Wait Classes by Total Wait Time). `ges resource hash list` latch: 2,048 misses, 2,024 sleeps — contentious. `enq: KO - fast object checkpoint` (background) = 3,997 waits, 21s, **17.36% of background time** — checkpoint coordination overhead between nodes.

Business impact: Currently minor (1.22% of DB Time). However, the `enq: KO - fast object checkpoint` background wait consuming 17.36% of background time indicates checkpoint coordination overhead which could impact recovery time and write performance under heavier load.

Confidence: Medium — RAC overhead is real but within acceptable ranges for a 2-node cluster; the `enq: KO` checkpoint contention is the more concerning signal for long-term health.

✅ Recommended Actions (prioritised)

🔴 CRITICAL — Create a Function-Based Index on TEST_SALES.CUSTOMER_NAME

What to do (plain): Add a "shortcut" (index) to the sales table that understands the `UPPER()` function — so Oracle can jump directly to matching rows instead of reading every single row 3,000 times.

What to do (technical):

```
-- Step 1: Check current indexes on the table (read-only investigation)
SELECT index_name, column_name, column_position, descend
FROM   dba_ind_columns
WHERE  table_owner = 'DEMO_USER'
AND    table_name  = 'TEST_SALES'
ORDER BY index_name, column_position;

-- Step 2: Check table size and row count
SELECT num_rows, blocks, last_analyzed
FROM   dba_tables
WHERE  owner = 'DEMO_USER' AND table_name = 'TEST_SALES';

-- Step 3: Create function-based index (DBA action - read-only recommendation)
-- CREATE INDEX demo_user.idx_test_sales_upper_cname
-- ON demo_user.test_sales (UPPER(customer_name), amount)
-- TABLESPACE users;

-- Step 4: Gather statistics on the new index
-- EXEC DBMS_STATS.GATHER_TABLE_STATS('DEMO_USER','TEST_SALES',cascade=>TRUE);
```

⚠️ Note: The CREATE INDEX and GATHER_STATS commands are DDL/DML-adjacent; they are provided as the specific technical action required — consult change management before executing in production.

Expected improvement: The full table scan (reading entire table) will be replaced by an index range scan. Buffer gets per execution should drop from **3,273** to **2–5**. Elapsed time for 3,000 executions should drop from **137.35s to under 1 second total**. DB CPU utilisation should drop from ~85% busy to near-idle.

Effort: 15–30 minutes (index build time depends on `TEST_SALES` row count — not available in AWR). Can be built `ONLINE` to avoid locking.

Risk: LOW — online index creation does not block DML. Statistics gather is non-blocking. **What this means:** Adding this index is like giving Oracle a phone book with an index at the back — instead of reading every page, it jumps straight to the right entry.

🔴 CRITICAL — Review and Refactor the run_bad_query PL/SQL Procedure

What to do (plain): The PL/SQL code calling the bad query in a loop of 10 should be reviewed — even after the index is added, looping the same query 1,000+ times per session invocation is architecturally wasteful and should be redesigned.

What to do (technical):

```
-- Identify the full text of run_bad_query procedure
SELECT text FROM dba_source
WHERE owner = 'DEMO_USER'   -- or appropriate schema
AND   name  = 'RUN_BAD_QUERY'
ORDER BY line;

-- Review the calling context of SQL_ID 9g4v8sh43hxdv
-- BEGIN FOR i IN 1..10 LOOP run_bad_query; END LOOP; END;
-- Questions to ask:
--   1. Why is COUNT(*) needed 10 times per loop?
--   2. Can the result be computed once and reused?
--   3. Can the WHERE clause use CUSTOMER_NAME directly (no UPPER) if data is stored consistently?

-- Check if data can be stored in a consistent case to avoid UPPER():
SELECT DISTINCT UPPER(customer_name) = customer_name AS already_upper,
       COUNT(*)
FROM   demo_user.test_sales
GROUP BY (UPPER(customer_name) = customer_name);
```

Expected improvement: If the query can be eliminated from the loop or called once with results cached in a PL/SQL variable, the 3,000 executions could be reduced to 1–10, reducing CPU by 95–99%.

Effort: 1–4 hours (code review + testing). Requires application team involvement.

Risk: MEDIUM — code change requires testing in non-production first. **What this means:** Even with the index fix, asking the database to run the same question 3,000 times in 11 minutes is wasteful — fixing the logic of the program will make it even faster and more scalable.

🟡 **HIGH — Investigate and Address Dictionary Cache Miss Rate (Row Cache Lock / Shared Pool)**

What to do (plain): The database's internal "index" of table definitions and metadata is missing too often — Oracle has to look up the same information repeatedly. Increasing the memory allocated to this cache, and ensuring SQL uses bind variables (placeholders), will reduce the frequency of these lookups.

What to do (technical):

```
-- 1. Check current shared pool and row cache usage
SELECT parameter, gets, getmisses,
       ROUND(getmisses/NULLIF(gets,0)*100,2) AS miss_pct
FROM   v$rowcache
WHERE  gets > 0
ORDER BY miss_pct DESC;

-- 2. Check for literal SQL (no bind variables) causing hard parses
SELECT force_matching_signature, COUNT(*) cnt,
       MAX(SUBSTR(sql_text,1,100)) sample_text
FROM   v$sql
WHERE  force_matching_signature ≠ 0
GROUP BY force_matching_signature
HAVING COUNT(*) > 5
ORDER BY cnt DESC;

-- 3. Check shared pool free memory trend
SELECT name, bytes/1024/1024 AS mb
FROM   v$sgastat
WHERE  pool = 'shared pool'
ORDER BY bytes DESC;

-- 4. If literal SQL is confirmed: enable cursor_sharing (assess impact first)
-- ALTER SYSTEM SET cursor_sharing = 'FORCE' SCOPE=BOTH;  -- evaluate carefully
-- ⚠️ cursor_sharing=FORCE can cause plan instability — test in non-production first

-- 5. Check if shared pool needs to grow (current: 288MB, usage at 73.86%)
-- Review: SGA Target = 960MB, consider redistributing if buffer cache is over-sized
SELECT component, current_size/1024/1024 AS current_mb,
       min_size/1024/1024 AS min_mb
FROM   v$sga_dynamic_components;
```

Expected improvement: Reducing `dc_histogram_defs` miss rate from 70% to <5% will reduce `row cache lock` waits (currently 1.89% DB Time) by an estimated 70–80%. Hard parse rate should also decrease.

Effort: 1–2 hours (investigation); parameter changes require monitoring for 24–48 hours.

Risk: LOW–MEDIUM — diagnostic steps are read-only; `cursor_sharing` change requires non-production validation. **What this means:** Making the database better at "remembering" what it already figured out will reduce the time spent relearning the same things over and over.

📋 **Action Checklist (copy-paste ready for DBA)**

- [ ] **IMMEDIATE: Identify TEST_SALES table size and row count** — run `SELECT num_rows, blocks FROM dba_tables WHERE owner='DEMO_USER' AND table_name='TEST_SALES'` — est. **5 min**
- [ ] **CRITICAL: Check existing indexes on TEST_SALES** — run `SELECT * FROM dba_indexes WHERE table_owner='DEMO_USER' AND table_name='TEST_SALES'` — est. **5 min**
- [ ] **CRITICAL: Create function-based index on UPPER(CUSTOMER_NAME), AMOUNT** — `CREATE INDEX ... ON demo_user.test_sales (UPPER(customer_name), amount)` — est. **15–60 min** (depends on table size; use ONLINE option)
- [ ] **CRITICAL: Gather fresh statistics after index creation** — `EXEC DBMS_STATS.GATHER_TABLE_STATS('DEMO_USER','TEST_SALES',cascade⇒TRUE)` — est. **5–30 min**
- [ ] **CRITICAL: Verify query now uses index** — run `EXPLAIN PLAN FOR SELECT COUNT(*) FROM demo_user.test_sales WHERE UPPER(customer_name)='CUSTOMER 500' AND amount > 500` and confirm INDEX RANGE SCAN — est. **5 min**
- [ ] **HIGH: Review run_bad_query procedure source code** — `SELECT text FROM dba_source WHERE name='RUN_BAD_QUERY'` — est. **30 min** (review + document findings)
- [ ] **HIGH: Assess whether the 10-iteration PL/SQL loop is intentional** — consult application developer; determine if COUNT(*) result changes between iterations — est. **1–2 hours**
- [ ] **MEDIUM: Investigate dictionary cache miss rates** — run `SELECT parameter, getmisses/gets*100 FROM v$rowcache WHERE gets>0 ORDER BY 2 DESC` — est. **15 min**
- [ ] **MEDIUM: Check for literal SQL (no bind variables)** — query `v$sql` for high `force_matching_signature` groups — est. **20 min**
- [ ] **MEDIUM: Monitor shared pool free memory** — currently at 56.98MB free (19.8% of 288MB shared pool); if it drops below 10%, plan a shared pool resize — est. **10 min** (monitoring)
- [ ] **LOW: Take a new AWR snapshot after fixes and compare** — `EXEC DBMS_WORKLOAD_REPOSITORY.CREATE_SNAPSHOT` before and after, then generate a new AWR report — est. **15 min**
- [ ] **LOW: Verify RAC interconnect health** — check `enq: KO - fast object checkpoint` background wait trend across both nodes — est. **20 min**

QUESTION

Tulis laporan insiden untuk manajemen berdasarkan AWR ini. Aturan:
- TIDAK ADA istilah teknis Oracle (tidak ada SQL_ID, "wait event", "buffer cache")
- Terjemahkan setiap temuan ke dampak bisnis (pengalaman pengguna, latensi transaksi, risiko pendapatan)
- Struktur: Ringkasan Eksekutif → Apa yang Terjadi → Dampak Bisnis → Akar Masalah (bahasa awam) → Rencana Perbaikan → Timeline → Estimasi Biaya
- Maksimal 400 kata total
- Akhiri dengan tabel 5 poin: Tindakan | Penanggung Jawab | Timeline | Risiko Jika Tidak Dilakukan | Status

Tulis dalam Bahasa Indonesia formal yang mudah dipahami oleh eksekutif.

LAPORAN INSIDEN PERFORMA SISTEM DATABASE

Klasifikasi: Internal — Manajemen | **Tanggal:** 17 Desember 2025 | **Tingkat Keparahan:** 🔴 Kritis

📋 Ringkasan Eksekutif

Pada pagi hari tanggal 17 Desember 2025, sistem database perusahaan terdeteksi mengalami beban kerja yang sangat tidak efisien. **Satu proses otomatis** — yang berjalan berulang kali dalam waktu singkat — menghabiskan **lebih dari 92% seluruh kapasitas komputasi database** hanya dalam 11 menit. Kondisi ini berpotensi membuat seluruh pengguna sistem mengalami kelambatan ekstrem apabila insiden serupa terjadi di jam kerja puncak.

🔍 Apa yang Terjadi

Sebuah program otomatis melakukan pencarian data ke tabel **Penjualan (TEST_SALES)** sebanyak **3.000 kali dalam 11 menit**. Setiap pencarian memaksa sistem membaca *seluruh isi tabel* dari awal hingga akhir — alih-alih langsung menuju data yang dicari — karena sistem tidak memiliki "panduan pencarian" (*shortcut*) yang semestinya dipasang sejak awal.

💼 Dampak Bisnis

Kondisi	Potensi Dampak
Jam sibuk (peak hour)	Seluruh transaksi pengguna melambat atau gagal
Proses laporan harian	Tertunda hingga berjam-jam
Kapasitas sistem	Terkuras 92% hanya oleh 1 proses, tidak tersisa untuk proses lain
Reputasi & pendapatan	Risiko downtime yang dapat memengaruhi kepuasan pelanggan dan SLA

🧩 Akar Masalah (Bahasa Awam)

Bayangkan sebuah perpustakaan besar tanpa katalog. Setiap kali staf ingin menemukan satu buku, mereka harus memeriksa **setiap rak, setiap buku, satu per satu** — lalu melakukan hal yang sama **3.000 kali berturut-turut**. Itulah yang terjadi: sistem tidak memiliki "katalog" (indeks data) untuk tabel Penjualan, dan program yang berjalan tidak dirancang secara efisien.

🛠️ Rencana Perbaikan

Pasang "katalog" data pada tabel Penjualan agar pencarian langsung menuju target — bukan membaca semua data.

Perbaiki logika program agar tidak memanggil pencarian yang sama ribuan kali secara sia-sia.

Pantau sistem secara berkelanjutan untuk mendeteksi pola serupa di masa depan.

⏱️ Timeline

```
Minggu 1 → Pasang katalog data (estimasi: 1 hari kerja)
Minggu 2 → Perbaikan & pengujian program (estimasi: 3-5 hari kerja)
Minggu 3 → Pemantauan & validasi hasil perbaikan
```

Estimasi Biaya Perbaikan: Rendah — dapat dikerjakan oleh tim internal tanpa biaya lisensi tambahan. **Estimasi Biaya Jika Tidak Diperbaiki:** Potensi *downtime* produksi + dampak SLA pelanggan.

✅ Tabel Tindakan Prioritas

Tindakan	Penanggung Jawab	Timeline	Risiko Jika Tidak Dilakukan	Status
Pasang indeks ("katalog") pada tabel Penjualan	Tim DBA	Minggu 1	Sistem lumpuh saat beban puncak	🔴 Belum Dimulai
Perbaiki logika program pencarian berulang	Tim Developer	Minggu 2	Pemborosan kapasitas server berlanjut	🔴 Belum Dimulai
Uji performa setelah perbaikan	Tim QA / DBA	Minggu 2–3	Perbaikan tidak tervalidasi	🟡 Menunggu
Pasang sistem peringatan dini (monitoring)	Tim Infrastruktur	Minggu 3	Insiden serupa tidak terdeteksi lebih awal	🟡 Menunggu
Tinjau ulang seluruh program sejenis	Manajer Aplikasi	Bulan 1	Risiko tersembunyi di proses lain	🟢 Direncanakan

Laporan ini disiapkan oleh Tim Database & Infrastruktur. Untuk pertanyaan teknis lebih lanjut, hubungi Kepala Tim DBA.

QUESTION

Create a complete AWR performance dashboard. Include all sections: stat_cards (6 key metrics), health (all areas with status + root_cause + action), top_events (all wait events with pct), top_sql (all SQL with flags and tuning_action), memory_stats, io_stats, rac_stats if applicable, and recommendations_priority_order. Make all values accurate numbers from the report — no strings for numeric fields.

DBRAC AWR Performance Dashboard

DBRAC / dbrac1 on rac1, Snap 28→29, 17-Dec-2025 10:14→10:25

● CRITICAL

DB TIME / ELAPSED
149.6s / 684.6s (21.8%)
AAS=0.22; light overall load but one SQL consumes 92% of all DB Time

DB CPU % OF DB TIME
83.53%
124.98s CPU of 149.6s DB Time — CPU-bound, driven by full table scan loop

WORST SQL % OF DB TIME
92.05%
SQL_ID 9g4v8sh43hxdv PL/SQL block: 3 execs x 1000 child calls = 3000 full scans

BUFFER CACHE HIT RATIO
99.99%
All 9,838,384 logical reads served from memory — disk is NOT the problem

LIBRARY CACHE HIT RATIO
87.59%
125 hard parses; dc_histogram_defs miss 70.46%; dc_sequences miss 100%

RAC BUSY+CONGESTED BLOCKS
19.28%
gc current block busy avg 10ms; enq:KO checkpoint = 17.36% of background time

Health Scorecard

10 areas monitored

CPU / Query Logic — CRITICAL

`DB CPU = 124.98s = 83.53% of DB Time; top SQL 92.05% of DB Time`

PL/SQL loop calls run_bad_query 1000x per execution x 3 executions = 3000 full table scans on TEST_SALES. UPPER(CUSTOMER_NAME) predicate is non-sargable — no function-based index exists. Each of 3000 executions reads 3,274 buffer blocks to return exactly 1 row.

```
$ CREATE INDEX demo_user.idx_ts_upper_cname ON demo_user.test_sales (UPPER(customer_name),
amount); then refactor PL/SQL to cache COUNT result outside the loop
```

Memory — Shared Pool — WARNING

`Library cache hit = 87.59%; shared pool free dropped from 69.26MB to 56.98MB (-12.28MB); hard parses = 125`

dc_histogram_defs miss rate 70.46% (350 misses of 497 gets) and dc_sequences miss rate 100% (4 of 4) indicate possible literal SQL or under-warmed dictionary cache. Shared pool still has 56.98MB free — not critically low but trending down.

→ Query v$rowcache for sustained miss rates. Check v$sql force_matching_signature for literal SQL patterns. If literals confirmed, evaluate cursor_sharing=FORCE in non-production first.

RAC Interconnect — WARNING

`gc current block busy: 45 waits avg 10.0ms; busy+congested = 19.28%; interconnect ping avg 8.79ms (500B) / 10.08ms (8K); enq:KO = 17.36% background time`

Foreground RAC overhead is only 1.22% of DB Time — acceptable. However 19.28% of cross-node block transfers are in busy or congested state, and enq:KO fast object checkpoint is consuming disproportionate background time, indicating checkpoint coordination overhead between nodes.

→ Monitor enq:KO trend on both nodes via v$event_histogram. Verify interconnect bandwidth. Review SYSAUX I/O placement. Fix CPU issue first — reduced workload will lower cross-node pressure.

Concurrency — Row Cache Lock — WARNING

`row cache lock: 1,257 waits, 2.80s total, avg 2.23ms, 1.89% DB Time`

High dc_histogram_defs miss rate (70.46%) is driving repeated row cache lock acquisitions for cross-node dictionary synchronisation in RAC. Not blocking user sessions but consuming DB Time.

```
$ Investigate: SELECT parameter, gethits, gets, getmisses, ROUND(getmisses/NULLIF(gets,0)*100,2)
miss_pct FROM v$rowcache WHERE parameter='dc_histogram_defs' ORDER BY getmisses DESC;
```

Parse / Cursor Sharing — WARNING

`Soft parse % = 80.71%; hard parses/txn = 17.86; parse CPU to elapsed = 29.13%`

Hard parse rate of 17.86 per transaction is elevated. Parse CPU at 29.13% of parse elapsed time suggests latch contention during parse. Likely caused by literal SQL values in the PL/SQL loop (CUSTOMER_NAME = 'CUSTOMER 500' hardcoded).

→ Rewrite run_bad_query to use bind variable: WHERE UPPER(customer_name) = :cname AND amount > :amt. This alone will drop hard parses to near zero.

Disk I/O — OK

`Physical reads = 845 total; db file sequential read = 0.32% DB Time; avg single-block read = 0.62ms`

TEST_SALES is fully cached in buffer cache — 9,821,800 logical reads against 0 physical reads for that table. All physical I/O is AWR/system overhead.

→ No action required. Re-validate physical read count after index creation to confirm TEST_SALES remains cached.

Memory — Buffer Cache — OK

`Buffer cache = 604MB; hit ratio = 99.99%; logical reads = 9,838,384 vs physical reads = 845`

SGA Target Advisor shows flat DB Time factor from 672MB to 1,152MB — increasing memory will not reduce DB Time until the CPU scan problem is fixed.

→ No sizing change needed. SGA = 960MB, memory_target = 1,600MB are adequate.

Concurrency — TX Row Lock — OK

`enq: TX row lock = 0 occurrences; user commits = 4 in snapshot`

Workload is read-only (COUNT queries). No DML contention. No blocking sessions.

→ No action required.

Commit / Redo Write — OK

`log file sync: 1 wait, 26ms avg, 0.02% DB Time; redo = 51,508 bytes total; redo writes = 7`

Near-zero commit activity. Redo and commit overhead are negligible.

→ No action required.

Undo / Snapshot Too Old Risk — OK

`ORA-01555 count = 0; tuned undo retention = 1,164s; max query length = 318s; undo blocks written = 43`

Auto-tuned retention (1,164s = 19.4 min) comfortably covers longest query (318s = 5.3 min). No ORA-01555 risk.

→ No action required. Monitor if long-running query duration grows beyond 1,164s.

Top Timed Events
% of total DB Time

Sorted by impact

SQL — Top Elapsed Time
5 statements

Sorted by elapsed

SQL ID	ELAPSED (S)	EXECS	ROWS / EXEC	MODULE · FLAGS
9g4v8sh43hxdv	137.68	—	—	SQL*Plus · PL/SQL loop amplifier: 3 … · 122.26s CPU = 88.8% of … · 0 physical reads — pure …
9u9w5ykpwumfs	137.35	—	1.0	SQL*Plus · UPPER(CUSTOMER_NA… · 3,274 buffer gets per ex… · 99.83% of all buffer gets…
54zzrhn169jst	3.76	—	—	MMON_SLAVE · 436 physical reads = 51.… · System background proc…
fnvwba7asyd7q	1.63	—	—	MMON_SLAVE · cluster_wait_s = 0.55s = … · System background proc…
0rgupjmzwfwn6	0.46	—	—	SQL*Plus · GV$ cross-instance quer… · Ad-hoc diagnostic use o…

Recommendations
4 actions

Ordered by priority

1 Create function-based index to eliminate full table scan

UPPER(CUSTOMER_NAME) predicate forces full table scan on TEST_SALES 3,000 times per 11-minute window — consuming 91.83% of all DB Time and 99.83% of all buffer gets

```
CREATE INDEX demo_user.idx_ts_upper_cname ON demo_user.test_sales (UPPER(customer_name), amount) ONLINE; EXEC DBMS_STATS.GATHER_TABLE_STATS('DEMO_USER','TEST_SALES',cascade=>TRUE);
```

↑ 95% improvement · ⏱ 0.25h effort · Risk: low

2 Refactor PL/SQL loop to eliminate 1000x redundant query executions

run_bad_query is called 1000 times per PL/SQL block execution. If COUNT result is static per invocation, all 999 extra executions are wasted — multiplying the index fix benefit by 1000x

```
Rewrite: cache result in variable before loop — v_count := run_bad_query(); then use v_count inside loop. Add bind variable: WHERE UPPER(customer_name) = :cname AND amount > :amt
```

↑ 99% improvement · ⏱ 1h effort · Risk: medium

3 Add bind variables to eliminate hard parses

Hard parse rate of 17.86 per transaction and library cache hit of 87.59% caused by hardcoded literal 'CUSTOMER 500' in SQL — each unique literal value forces a full recompile

```
Replace literal in run_bad_query: WHERE UPPER(customer_name) = :p_cname AND amount > :p_amount. Verify with: SELECT force_matching_signature, COUNT(*) FROM v$sql GROUP BY force_matching_signatur
e HAVING COUNT(*)>10;
```

↑ 80% improvement · ⏱ 0.5h effort · Risk: low

4 Investigate RAC enq:KO checkpoint and interconnect latency spikes

enq:KO fast object checkpoint consuming 17.36% of background time and interconnect ping spiking to 101-102ms (vs 8-10ms average) indicate periodic RAC coordination storms

```
SELECT event, total_waits, time_waited_micro/1000000 wait_s FROM v$system_event WHERE event LIKE 'enq: KO%'; review interconnect NIC errors via /proc/net/dev on both nodes; check SYSAUX I/O late
ncy
```

↑ 15% improvement · ⏱ 2h effort · Risk: low

Oracle AI Analyst · 4/6/2026, 6:38:58 PM

19,138 tokens · $0.1484

Copy · ↓ PDF

QUESTION

Buat dashboard ringkasan AWR ini

DBRAC AWR Dashboard

DBRAC / dbrac1 on rac1, Snap 28→29, 17-Dec-2025 10:14→10:25

● CRITICAL

DB TIME / ELAPSED

149.6s / 684.6s (21.8%)

AAS=0.22; ringan secara keseluruhan, tapi 1 SQL menyedot 92% DB Time

DB CPU % OF DB TIME

83.53%

124.98s dari 149.6s DB Time adalah CPU — bukan I/O yang jadi masalah

SQL TERBURUK % DB TIME

92.05%

SQL_ID 9g4v8sh43hxdv: PL/SQL loop 3 eksekusi x 1000 panggilan = 3000 full scan

BUFFER CACHE HIT RATIO

99.99%

9,838,384 logical reads dari memori — disk bukan penyebab masalah

LIBRARY CACHE HIT RATIO

87.59%

125 hard parse; dc_histogram_defs miss 70.46%; literal SQL tanpa bind variable

RAC BUSY+CONGESTED BLOCK

19.28%

gc current block busy avg 10ms; enq:KO = 17.36% background time

Health Scorecard 6 areas monitored

CPU / Query Logic — CRITICAL

DB CPU 124.98s = 83.53% DB Time; SQL terburuk 92.05% DB Time

Loop PL/SQL memanggil full table scan TEST_SALES 3000x. UPPER(CUSTOMER_NAME) membuat index tidak bisa dipakai. Setiap eksekusi membaca 3,274 blok hanya untuk mengembalikan 1 baris.

→ Buat function-based index pada UPPER(customer_name); refactor PL/SQL loop; tambahkan bind variable

Shared Pool / Parse — WARNING

Soft parse 80.71%; hard parse/txn = 17.86; library cache hit 87.59%

Literal SQL hardcoded ('CUSTOMER 500') menyebabkan hard parse berulang. dc_histogram_defs miss 70.46% mendorong row cache lock di RAC.

→ Ganti literal dengan bind variable (:p_cname, :p_amount); periksa v$rowcache untuk dc_histogram_defs

RAC Interconnect — WARNING

Cluster wait 1.22% DB Time; busy+congested 19.28%; ping max spike 101ms

Overhead foreground RAC masih 1.22% — dapat diterima. Namun enq:KO menyedot 17.36% background time dan ping sesekali melonjak ke 101-102ms dari rata-rata 8-10ms.

→ Monitor enq:KO di v$system_event; periksa NIC error di kedua node; selesaikan CPU issue terlebih dahulu

Disk I/O — OK

Physical reads = 845; db file sequential read avg = 0.62ms; 0.32% DB Time

TEST_SALES sepenuhnya di-cache di buffer pool — 0 physical reads untuk tabel itu. Semua I/O fisik adalah overhead AWR/system.

→ Tidak perlu tindakan. Validasi ulang setelah index dibuat.

Memory — OK

Buffer cache 604MB; hit ratio 99.99%; shared pool free turun 69MB → 57MB

SGA Advisor menunjukkan DB Time tidak berubah dari 672MB hingga 1,152MB — menambah memori tidak akan membantu sebelum masalah CPU scan diselesaikan.

→ Tidak perlu resize. Selesaikan masalah CPU terlebih dahulu.

Commit / Redo / Undo — OK

log file sync: 1 wait 26ms; redo 51,508 bytes; ORA-01555 = 0; undo retention 1,164s

Workload hampir sepenuhnya read-only. Commit dan undo overhead sangat kecil. Tidak ada risiko Snapshot Too Old.

→ Tidak perlu tindakan.

Top Timed Events % of total DB Time

Sorted by impact

1	DB CPU	CPU	83.5% — 83.5% CRITICAL
2	row cache lock	Concurrency	1.9% HIGH
3	PL/SQL execution elapsed …	CPU	1.3% MEDIUM
4	db file sequential read	I/O	0.3% LOW
5	gc current block b…	Concurrency	0.3% NORMAL
6	gc current block 2…	Concurrency	0.3% NORMAL
7	gc current grant b…	Concurrency	0.2% NORMAL
8	SQL*Net message from c…	Network	0.1% NORMAL
9	log file sync	Commit	0.0% NORMAL
10	SQL*Net message to cli…	Network	0.0% NORMAL

I/O · CPU · Commit · Memory · Concurrency · Network

SQL — Top Elapsed Time 5 statements

Sorted by elapsed

SQL ID	ELAPSED (S)	EXECS	ROWS / EXEC	MODULE · FLAGS
9g4v8sh43hxdv	137.68	3	0 !	SQL*Plus · PL/SQL loop: 3 parent x 1... · 122.26s CPU = 88.8% se... · 0 physical reads — murn...
9u9w5ykpwumfs	137.35	3,000	1.0	SQL*Plus · UPPER(CUSTOMER_NA... · 3,274 buffer gets per ek... · 99.83% dari seluruh buff...
54zzrhn169jst	3.76	1	0 !	MMON_SLAVE · 436 physical reads = 51.... · AWR snapshot system o...
fnvwba7asyd7q	1.63	1	0 !	MMON_SLAVE · cluster_wait = 0.55s = 3... · AWR snapshot system o...
0rgupjmzwfwn6	0.46	2	0 !	SQL*Plus · GV$ cross-instance quer... · Query diagnostik ad-hoc...

Recommendations 4 actions

Ordered by priority

1 Buat function-based index untuk eliminasi full table scan

UPPER(CUSTOMER_NAME) memaksa full scan TEST_SALES 3,000x per 11 menit — menyedot 91.83% DB Time dan 99.83% buffer gets

```
CREATE INDEX demo_user.idx_ts_upper_cname ON demo_user.test_sales (UPPER(customer_name), amount) ONLINE;
```

↑ 95% improvement · ⏱ 0.25h effort · Risk: low

2 Refactor PL/SQL loop — eliminasi 1000x eksekusi redundan

run_bad_query dipanggil 1000x per blok PL/SQL. Jika hasil COUNT statis per pemanggilan, 999 eksekusi berikutnya adalah pemborosan murni

```
Cache hasil sebelum loop: v_count := run_bad_query(); gunakan v_count di dalam loop. Tambah bind variable: WHERE UPPER(customer_name) = :cname AND amount > :amt
```

↑ 99% improvement · ⏱ 1h effort · Risk: medium

3 Tambahkan bind variable untuk eliminasi hard parse

Literal hardcoded 'CUSTOMER 500' menyebabkan hard parse 17.86x per transaksi dan library cache hit turun ke 87.59%

```
Ganti literal di run_bad_query: WHERE UPPER(customer_name) = :p_cname AND amount > :p_amount. Verifikasi: SELECT force_matching_signature, COUNT(*) FROM v$sql GROUP BY force_matching_signature H
AVING COUNT(*)>10;
```

↑ 80% improvement · ⏱ 0.5h effort · Risk: low

4 Investigasi RAC enq:KO dan lonjakan latency interconnect

enq:KO menyedot 17.36% background time dan ping interconnect sesekali melonjak ke 101-102ms (vs rata-rata 8-10ms)

```
SELECT event, total_waits, time_waited_micro/1e6 FROM v$system_event WHERE event LIKE 'enq: KO%'; periksa NIC error di /proc/net/dev kedua node RAC
```

↑ 15% improvement · ⏱ 2h effort · Risk: low

Oracle AI Analyst · 4/6/2026, 6:38:58 PM

14,198 tokens · $0.0815

Copy · ↓ PDF